

Pillar 5: Capstone Project

Submitted by: **Darwin A. Nepomuceno**

Post Graduate Diploma in Artificial Intelligence and Machine Learning

Project Pipeline: Automated Early Detection of Fetal Distress

Focus: Cost-Sensitive Learning, Explainability (XAI), and Clinical Validity.

Project Setup & Environment

Goal: Establish a reproducible and organized foundation.

Directory Structure:

```
/AIM CAPSTONE PROJECT
├── /00 project information
│   ├── 01 Capstone Project-Automated Early Detection of Fetal
│   │   │   Distress.pdf
│   └── 02 Capstone Project-Automated Early Detection of Fetal
│       │   Distress-Pipeline.pdf
├── /01 data
│   ├── raw/ (Original CSV)
│   ├── clean/ (Cleaned CSVs)
│   └── sample_input_files/
├── /02 src
│   ├── .github/
│   │   ├── workflows/
│   │   └── ci.yaml
│   ├── api/
│   │   └── cmain.py
│   ├── config/
│   │   └── config.yaml
│   ├── models/
│   │   └── model_artifact.pkl
```

```

├── tests
│   └── test_api.py
├── wandb/
├── __init__.py
├── app.py
├── build_artifact.py
├── data_loader.py
├── Dockerfile
├── genai_tools.py
├── model.py
├── pipeline.py
├── preprocessor.py
├── wandb_track.py
└── requirements_app.txt

/03 notebooks
├── PHASE 01 CTG Exploratory Data Analysis.ipynb
├── PHASE 02 CTG Data Preprocessing.ipynb
├── PHASE 03 CTG Baseline & Cost-Sensitive Modeling.ipynb
├── PHASE 04 CTG Threshold Optimization & Validation.ipynb
├── PHASE 05 CTG Explainability (XAI) & Clinical
Validation.ipynb
├── PHASE 06 CTG Ethical AI and Bias Auditing.ipynb
├── PHASE 07 CTG Deployment (Interactive Tool).ipynb
├── PHASE 08 CTG Deployment & MLOps.ipynb
├── PHASE 09 CTG Documentation & Reporting.ipynb
└── PHASE 09 CTG Full Pipeline Demo.ipynb

/04 misc
├── ctg_preprocessed.pkl
└── deployment_artifact.pkl

/05 reports
├── Deck-Business.ppt
├── Deck-Technical.ppt
└── Final Report.pdf

/06 instructions
├── PHASE 07 CTG Instructions 00-Package Usage.pdf
├── PHASE 07 CTG Instructions 01-Deployment (Interactive
Tool).pdf
├── PHASE 07 CTG Instructions 02-GenAI-Enhanced Interface
Documentation.pdf
└── PHASE 08 CTG Instructions 03-Deployment & MLOps.pdf

/07 demo videos
├── Demo Video 01-FetalGuard AI App.mp4
└── Demo Video 02-Tracking in Weights & Biases.mp4

requirements.txt

```

Environment:

```
* Create `venv`: `python -m venv venv`
* Install packages:
  ```
 pip install pandas numpy scikit-learn xgboost imbalanced-
 learn shap matplotlib seaborn plotly streamlit optuna
  ```
```

Data Loading:

```
* Load the dataset.
* **Check:** Verify column names match the standard UCI/Kaggle
  schema (e.g., `Baseline FHR`, `AC`, `DC`, `Variance`, etc.).
```

Phase 01: Exploratory Data Analysis (EDA) & Cleaning

Goal: Understand data quality and distribution without leakage.

1. Initial Inspection:

- Check for **NaN** values. If present, impute with **Median** (robust to outliers) or **Mode**.
- Check for duplicates and remove them.

2. Target Variable Engineering:

- **Action:** Map **1** (Normal) → **0** (Normal), **2** (Suspect) & **3** (Pathological) → **1** (Abnormal).
- **Validation:** Plot distribution. Confirm the imbalance ratio (e.g., 75% Normal vs. 25% Abnormal).

3. Feature Analysis:

- **Correlation Matrix:** Heatmap to identify highly correlated features (>0.9).
 - *Strategy:* **Do not drop yet.** Note them for later model-specific handling.
- **Outlier Check:** Boxplots for key features.
 - *Action:* Cap extreme outliers (Winsorization at 1st and 99th percentile) if they are physiologically impossible (e.g., FHR > 250).

4. Statistical Testing:

- Run **Chi-Square** or **ANOVA** tests to see which features have a statistically significant difference between Normal and Abnormal groups.
-

Phase 02: Preprocessing & The "No-Leakage" Pipeline

Goal: Prepare data strictly avoiding data leakage.

1. Strict Splitting:

- **CRITICAL:** Split data **BEFORE** any scaling or resampling.
- `X_train, X_test, y_train, y_test = train_test_split(..., stratify=y, test_size=0.2, random_state=42)`

2. Scaling:

- Initialize `StandardScaler` .
- `scaler.fit(X_train)` → `X_train_scaled = scaler.transform(X_train)`
- `X_test_scaled = scaler.transform(X_test)` (Never fit on test data).

3. Resampling Strategy (Training Set Only):

- **Option A (SMOTE):** `smote = SMOTE(random_state=42);`
`X_train_resampled, y_train_resampled =`
`smote.fit_resample(X_train_scaled, y_train)`
- **Option B (ADASYN):** Compare with SMOTE.
- **Option C (Class Weights):** Keep raw training data but pass `class_weight='balanced'` to the model.
- **Note:** The `X_test` remains **unsampled** and unmodified to represent real-world distribution.

Phase 03: Baseline & Cost-Sensitive Modeling

Goal: Build models optimized for "Clinical Cost" rather than Accuracy.

1. Baseline Models (Imbalanced Data):

- Train **Logistic Regression** and **Decision Tree** on raw `X_train_scaled` (no SMOTE).
- Evaluate: Accuracy, Precision, Recall, F1.
- *Expectation:* High Accuracy, Low Recall for "Abnormal".

2. Advanced Models (With Strategies):

- Train **Random Forest**, **XGBoost**, and **LightGBM**.
- **Strategy 1:** Train on SMOTE data.
- **Strategy 2:** Train on raw data with `scale_pos_weight` (XGBoost) or `class_weight` (RF).

3. Custom Cost Metric Implementation:

- Define the cost function:
 - $Cost = (1 \times FP) + (10 \times FN)$
- Create a **Scorer Wrapper** for `GridSearchCV` :


```
from sklearn.metrics import make_scorer, confusion_matrix

def cost_scorer(y_true, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    return -1 * ((fp * 1) + (fn * 10)) # Negative because
    sklearn minimizes
```



```
custom_score = make_scorer(cost_scorer, greater_is_better=False)
```

4. Hyperparameter Tuning:

- Use `Optuna` or `GridSearchCV` with `scoring=custom_score` .
- **Goal:** Minimize the Total Expected Cost.

Phase 04: Threshold Optimization & Validation

Goal: Find the optimal decision boundary.

1. **Cross-Validation:**

- Perform **Stratified 5-Fold CV** on the best model.
- Report mean/std of Recall and Cost.

2. **Threshold Tuning:**

- Get probability predictions: `y_proba = model.predict_proba(X_test)[: , 1]`
- Iterate thresholds from 0.1 to 0.9:
 - Calculate Cost for each threshold.
 - **Select Threshold** where Cost is minimized (or Recall is >95% with max Precision).
- *Output:* A plot showing Cost vs. Threshold to justify your chosen value (e.g., 0.35).

3. **Final Evaluation:**

- Apply the optimal threshold to `X_test`.
 - Generate final **Confusion Matrix**, **ROC Curve**, and **Precision-Recall Curve**.
-

Phase 05: Explainability (XAI) & Clinical Validation

Goal: Prove the model makes sense to doctors.

1. **SHAP Implementation:**

- Initialize `shap.TreeExplainer(model)`.
- Compute SHAP values on `X_test` (or a sample).

2. **Visualizations:**

- **Summary Plot:** Show global feature importance.
- **Force Plot:** Explain a specific "False Positive" and a "True Positive" case.

3. **Clinical Alignment Check:**

- **Task:** Cross-reference top SHAP features with medical literature.
 - *Example:* "The model correctly identified 'Decelerations' and 'Variability' as top drivers, consistent with FIGO guidelines."
 - *Report:* Include a table mapping "Model Feature Importance" vs. "Clinical Relevance".
-

Phase 06: Ethical AI & Bias Auditing

Goal: Critically examine the model through four lenses: explainability, limitations, fairness, and proposed mitigations.

A model that performs well on aggregate metrics can still be harmful if it behaves inconsistently across patient groups or relies on clinically

inappropriate signals. This phase surfaces those risks using both XAI methods and formal fairness metrics.

Section	Focus
1	SHAP — already computed in Phase 05, reviewed here in a limitations context
2	LIME — local surrogate explanations for individual predictions
3	PDP — global feature effect across its entire value range
4	ICE — per-patient heterogeneity within the global PDP trend
5	Model Limitations — imbalance, leakage risk, overfitting, simplification
6	Bias Audit — proxy group analysis + formal fairness metrics via AIF360
7	Mitigation Proposals — actionable remediation strategies

Phase 07: Deployment (Interactive Tool)

Goal: Create a user-friendly interface for stakeholders.

1. Streamlit App (`app.py`):

- **Input:** Sliders/Input fields for the top 5 SHAP features (e.g., `FHR` , `Variability` , `Decelerations`).
- **Logic:**
 - Preprocess input (scale using the saved scaler).
 - Predict probability.
 - Apply the **Optimal Threshold** (e.g., 0.35).
- **Output:**
 - **Prediction:** "Normal" or "High Risk".
 - **Confidence:** Probability score.
 - **Explanation:** "Risk is high primarily due to **Low Variability** and **High Decelerations**."
- **Advanced Feature:** Add a "Sensitivity Slider" allowing the user to see how the prediction changes if they demand higher safety (lower threshold).

2. Hosting:

- Push to GitHub.
- Deploy to **Streamlit Cloud** (connect repo, auto-deploy).

Phase 08: Deployment Guide & MLOps Practices

Phase 09: Documentation & Reporting

Goal: Compile the narrative.

1. Report Structure:

- **Abstract:**
- **Methodology:** Detail the "No-Leakage" pipeline and the Custom Cost Function.
- **Results:**
 - Comparison of SMOTE vs. Class Weights.
 - Confusion Matrix with the **Optimal Threshold**.
 - Cost reduction achieved vs. Baseline.
- **Discussion:**
 - Limitations (e.g., dataset size, synthetic nature).
- **Conclusion:** Summary of impact.

2. Code Quality:

- Ensure all code is modular (`02 src/` folder).
 - Add docstrings and comments explaining *why* a step was taken (e.g., "Scaling applied after split to prevent leakage").
-